

Injecting Faults Along an Apollo Descent

Study H — 12 engine plants x 15 injection points on one nominal trajectory

22 August 2026

Contents

1	What this study does	1
2	The nominal descent	2
2.1	Where the initial state comes from	2
2.2	Two things the reference trajectory needed	2
2.3	The result	3
3	The injection experiment	3
3.1	Making sure the failures are real	4
4	Results	4
4.1	The outcome grid	4
4.2	What each fault costs	5
4.3	Survival against injection time	6
4.4	How much gate margin is left	6
4.5	The recoveries themselves	8
5	What this means	8
6	Limitations	8
7	Reproducing	9

1 What this study does

Study G sampled initial conditions from boxes: wide regions of the state space, covered quasi-randomly. That answers “from how much of the reachable space is this fault survivable”, but it does not answer the question a mission analyst actually asks, which is **where on the descent can this fault be survived**.

This study asks that one. It computes a single **nominal descent** for a healthy vehicle, then walks along it and injects each fault at 15 points. Because the nominal is a trajectory, every injection point comes with a time, an altitude, a range and a speed — so the result is a function of *when*, not a statistic over a box.

Three things change from Study G:

- **The engines move to $y = \pm 0.25$ m** from ± 1.50 m. This is the roll-authority threshold established in Study A: a single gimbal can trim a one-engine-out asymmetry only while $y_{eng} \leq dz_{eng} \tan \delta_{max} = 0.263$ m. At 1.50 m the vehicle could never survive an engine failure and the fault was uninformative — 0 landings from every state tested. At 0.25 m it sits just inside the limit, so engine-out becomes a fault the study can actually measure.
- **The nominal is anchored on the real Apollo approach geometry** rather than on the earlier studies’ near-vertical scenario (see below).

- **The gimbal-actuator faults are removed**, as requested: bandwidth loss, light damping, seizure and effectiveness loss are all about the gimbal’s own response, which is not this study’s subject. Thrust-vector misalignment is kept — it is a fixed nozzle offset from asymmetric erosion, present even if the gimbal were welded solid, and it is the only remaining additive fault besides chugging.

2 The nominal descent

2.1 Where the initial state comes from

The anchor is taken from the published Apollo approach-phase (P64) geometry rather than invented. Two points of that phase are well documented:

Point	Altitude	Range to target	Horizontal speed	Descent rate
high gate	7,500 ft (2,286 m)	4.5 nmi (8,300 m)	500 ft/s (152 m/s)	145 ft/s (44 m/s)
low gate	500 ft (152 m)	2,000 ft (610 m)	60 ft/s (18 m/s)	16 ft/s (5 m/s)

The approach between them is flown essentially straight at the landing point — the line-of-sight depression is 15.4° from high gate and 14.0° from low gate, the same path. Interpolating along it gives a one-parameter family of Apollo-consistent states, and this study takes the point where the horizontal speed has bled to 55 m/s, which is the fastest state this vehicle model’s own path constraint (60 m/s) admits with margin:

Anchor state	SI	Imperial
Altitude	736 m	2416 ft
Range to pad	2724 m	8937 ft
Horizontal speed	55.0 m/s	180 ft/s
Descent rate	15.6 m/s	51 ft/s
Line-of-sight depression	15.13°	
Flight-path angle	15.87°	

The last two lines are the consistency check that matters. A vehicle whose velocity vector points at the landing site is what “flying the approach phase” means, and here the line-of-sight depression and the flight-path angle agree to 0.75° — which they would not if the interpolation were nonsense.

This is a reconstruction of the geometry, not telemetry. It reproduces where an Apollo LM was and how fast it was going at 736 m on final approach; it is not a replay of any specific mission’s recorded state vector.

2.2 Two things the reference trajectory needed

Neither is cosmetic, and both are worth recording because the first attempt produced a trajectory that was not a lunar descent at all.

A spherical speed cap. The vehicle model’s velocity limit is a *per-axis* box, so a vehicle riding it on all three axes reaches 104 m/s. On the earlier studies’ near-vertical descent that is harmless — only the vertical channel is ever large. On a 2.7 km shallow approach it is not: the first solve accelerated downrange to 104 m/s, rode the 45° pitch limit, overshoot the pad and came back to it. The fix is a cap on the speed *norm*, set to 60 m/s. It is a new optional field on the problem configuration and defaults to off, so every earlier study is unaffected.

A fitted horizon. Surplus planning time is not free. A first solve on a 120 s horizon reached the contact altitude at 63 s and then held 1 m altitude for the remaining 57 s; a 72 s horizon still left the vehicle wandering ~70 m around the pad for the last 25 s, because the optimiser had time to spend and spent it. The horizon is therefore *measured*: a probe solve times the descent, and the trajectory is re-solved on a horizon just long enough to contain it — here 68 s.

The glide-slope floor is also relaxed from 30° to 12° . Apollo’s real approach is a $\sim 15^\circ$ path, which a 30° cone forbids outright; the old value was chosen for a much steeper, closer-in scenario.

2.3 The result

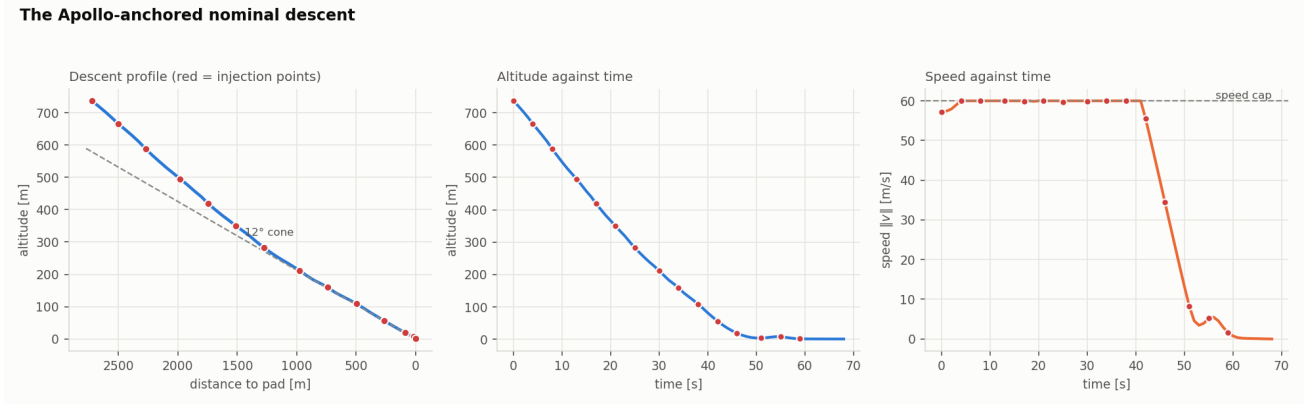


Figure 1: The Apollo-anchored nominal descent

The healthy vehicle cruises at the speed cap down the 15° path, brakes from about 40 s, reaches the contact altitude at $t = 65$ s and touches down on the pad with a gate margin of **0.60** — comfortably inside every landing criterion.

One honest caveat on the profile: the real P64 approach decelerates continuously from 152 m/s to 18 m/s, whereas this trajectory cruises at the cap and brakes late. That is the optimiser’s doing — its cost penalises distance to the target at every node, so arriving sooner is cheaper — not a property of Apollo. The *geometry* is Apollo-anchored; the speed schedule along it is the planner’s.

3 The injection experiment

At each of the 15 injection points the state is read straight off the nominal — all 22 of them, including the thrust and gimbal states the descent had the vehicle holding. The plant then becomes the damaged plant from that instant onward, the vehicle re-plans over the time the nominal had left plus a 20 s reserve, and the touchdown is scored against the Apollo landing gate (vertical speed ≤ 3.0 m/s, horizontal ≤ 1.2 m/s, tilt $\leq 6^\circ$, within 15 m of the pad, rates $\leq 5^\circ/\text{s}$).

Point	Injection time	Altitude	Plants that landed
0	0 s	736 m	12/12
1	4 s	666 m	12/12
2	8 s	587 m	12/12
3	13 s	495 m	12/12
4	17 s	419 m	11/12
5	21 s	350 m	12/12
6	25 s	282 m	11/12
7	30 s	212 m	11/12
8	34 s	160 m	12/12
9	38 s	109 m	10/12
10	42 s	56 m	10/12
11	46 s	19 m	9/12
12	51 s	4 m	11/12
13	55 s	8 m	11/12
14	59 s	2 m	12/12

The healthy plant is run from every injection point as the control. Because each state is taken from a trajectory that lands, it **must** land from every point, and it does — all 15 of them. Any failure elsewhere in the grid is therefore the fault, not the re-planning problem.

3.1 Making sure the failures are real

A “no trajectory found” is weaker evidence than a landing. The NLP is nonconvex, so a converged solve *proves* a trajectory exists, while a failed one only says the solver did not find one from its seed. The campaign’s own diagnostics said that mattered here: several failures gave up after 23-60 IPOPT iterations out of a 400 budget, sitting between neighbours that converged in 160-207 — the signature of a bad local corner rather than a boundary.

Every `no_recovery` was therefore re-attacked with six horizon seeds at 1,200 iterations each. Extra search can only turn a false “no” into a true “yes”, so spending it on the failures alone reduces false negatives without being able to manufacture a landing. **None of the failures flipped.** All of them survived every seed, so the boundaries reported below are the vehicle’s, not the solver’s.

What is deliberately not modelled is the reaction gap — the seconds between the fault occurring and the guidance responding, during which the vehicle flies a stale command on a broken engine. Detection here is instantaneous and the damaged plant is known exactly. That gap is Study D/E’s subject; leaving it out makes this a clean measurement of *where on the descent* a fault is survivable at all, and an upper bound on what any real system achieves.

4 Results

4.1 The outcome grid

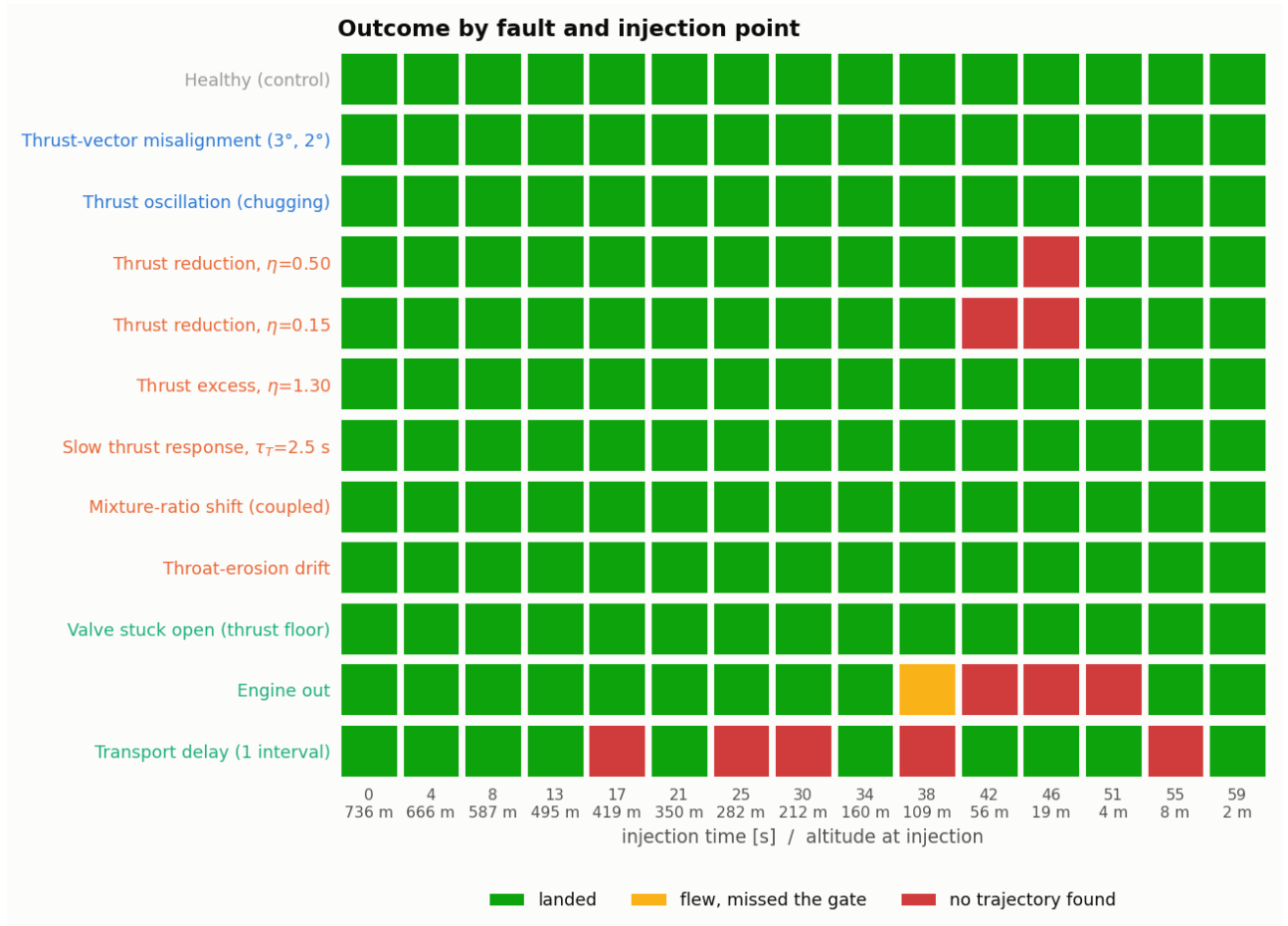


Figure 2: Outcome by fault and injection point

Fault	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	landed
Healthy (control)	L	L	L	L	L	L	L	L	L	L	L	L	L	L	L	15/15
Thrust-vector misalignment (3° , 2°)	L	L	L	L	L	L	L	L	L	L	L	L	L	L	L	15/15
Thrust oscillation (chugging)	L	L	L	L	L	L	L	L	L	L	L	L	L	L	L	15/15
Thrust reduction, $\eta=0.50$	L	L	L	L	L	L	L	L	L	L	.	L	L	L	L	14/15
Thrust reduction, $\eta=0.15$	L	L	L	L	L	L	L	L	L	L	.	.	L	L	L	13/15
Thrust excess, $\eta=1.30$	L	L	L	L	L	L	L	L	L	L	L	L	L	L	L	15/15
Slow thrust response, $\tau_T=2.5$ s	L	L	L	L	L	L	L	L	L	L	L	L	L	L	L	15/15
Mixture-ratio shift (coupled)	L	L	L	L	L	L	L	L	L	L	L	L	L	L	L	15/15
Throat-erosion drift	L	L	L	L	L	L	L	L	L	L	L	L	L	L	L	15/15
Valve stuck open (thrust floor)	L	L	L	L	L	L	L	L	L	L	L	L	L	L	L	15/15
Engine out	L	L	L	L	L	L	L	L	L	g	.	.	.	L	L	11/15
Transport delay (1 interval)	L	L	L	L	.	L	.	.	L	.	L	L	L	.	L	10/15

L landed · g flew but missed the gate · . no trajectory found

4.2 What each fault costs

Fault	Class	Landing rate	k/n	Latest survivable injection
Healthy (control)	none	1.00 [0.80, 1.00]	15/15	every point
Thrust-vector misalignment (3° , 2°)	additive	1.00 [0.80, 1.00]	15/15	every point
Thrust oscillation (chugging)	additive	1.00 [0.80, 1.00]	15/15	every point
Thrust excess, $\eta=1.30$	multiplicative	1.00 [0.80, 1.00]	15/15	every point
Slow thrust response, $\tau_T=2.5$ s	multiplicative	1.00 [0.80, 1.00]	15/15	every point
Mixture-ratio shift (coupled)	multiplicative	1.00 [0.80, 1.00]	15/15	every point
Throat-erosion drift	multiplicative	1.00 [0.80, 1.00]	15/15	every point
Valve stuck open (thrust floor)	structural	1.00 [0.80, 1.00]	15/15	every point
Thrust reduction, $\eta=0.50$	multiplicative	0.93 [0.70, 0.99]	14/15	59 s / 2 m
Thrust reduction, $\eta=0.15$	multiplicative	0.87 [0.62, 0.96]	13/15	59 s / 2 m
Engine out	structural	0.73 [0.48, 0.89]	11/15	59 s / 2 m
Transport delay (1 interval)	structural	0.67 [0.42, 0.85]	10/15	59 s / 2 m

7 of the 11 faults land from every injection point (Thrust-vector misalignment (3° , 2°), Thrust oscillation (chugging), Thrust excess, $\eta=1.30$, Slow thrust response, $\tau_T=2.5$ s, Mixture-ratio shift (coupled), Throat-erosion drift, Valve stuck open (thrust floor)). Given the whole descent to choose from, these never take the vehicle out — the planner absorbs them wherever they arrive.

The interesting 4 are the ones with a **boundary** — survivable early, fatal late:

Fault	Last survivable t	at altitude	First fatal t	at altitude
Thrust reduction, $\eta=0.50$	59 s	2 m	46 s	19 m
Thrust reduction, $\eta=0.15$	59 s	2 m	42 s	56 m
Engine out	59 s	2 m	38 s	109 m
Transport delay (1 interval)	59 s	2 m	17 s	419 m

That boundary is the operationally useful number in this whole study: it is the point on the descent past which the fault must be prevented rather than recovered from, because after it no guidance law helps.

One row should not be read that way. **Transport delay fails in a scattered pattern rather than at a boundary** — it loses points spread across the descent and lands between them. That survived the hardening pass, so it is not solver noise, but a non-monotone pattern in injection time is not what a physical authority limit looks like. The likely reading is that a delay of exactly one control interval interacts with the discretisation — the replan horizon changes by a few steps from point to point, and a whole-interval lag lands differently against it each time. Dead time is the one fault in this catalogue whose natural scale (10-30 ms, per the framework) is far below what a 1 s planning grid can represent, so this row measures the model’s grid as much as the vehicle. No boundary is quoted for it.

4.3 Survival against injection time

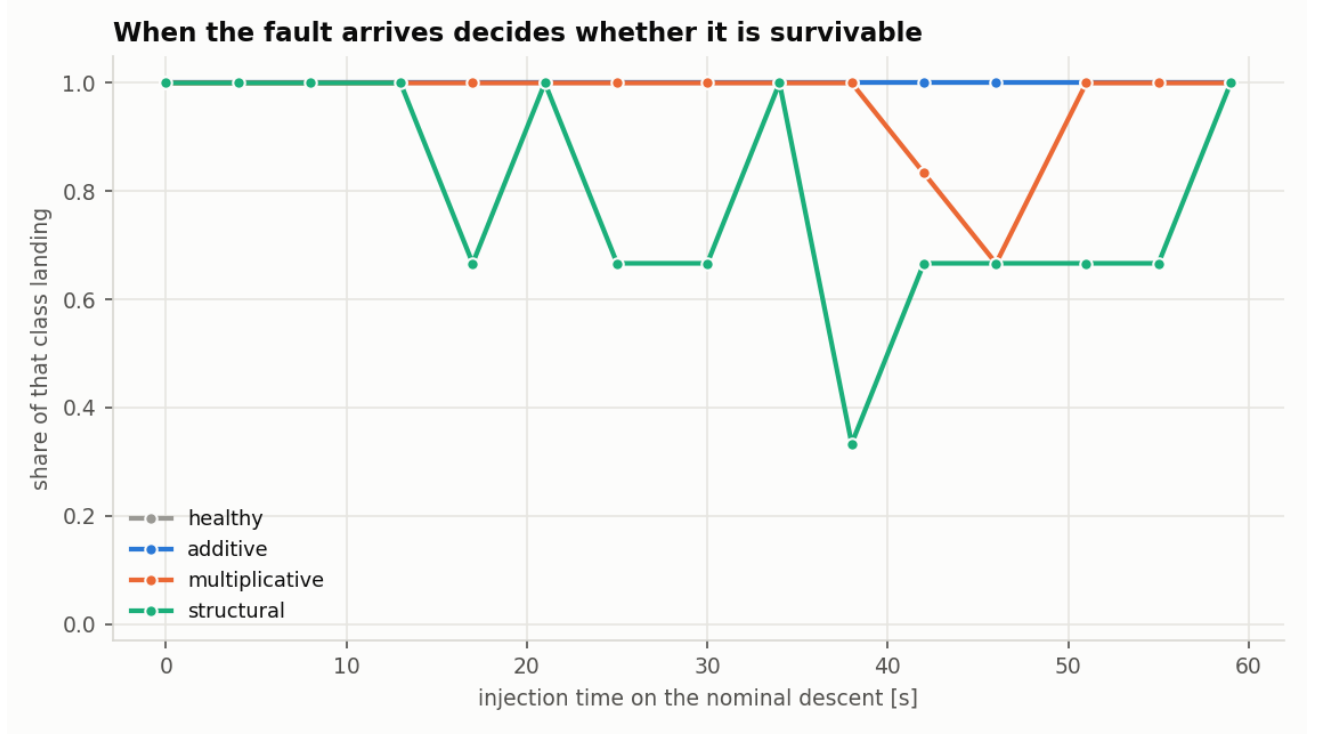


Figure 3: Landing share by fault class against injection time

Fault class	Landing rate	k/n
healthy	1.00	15/15
additive	1.00	30/30
multiplicative	0.97	87/90
structural	0.80	36/45

4.4 How much gate margin is left

Landing is a binary; the margin behind it is not. A margin of 1.0 *is* the gate, and the distance below it is how much of the touchdown budget the recovery had to spend — which is the early warning that a fault is approaching its boundary.

Touchdown gate margin against injection time (dashed line = the gate; points above it miss; a point at the ceiling found no trajectory)

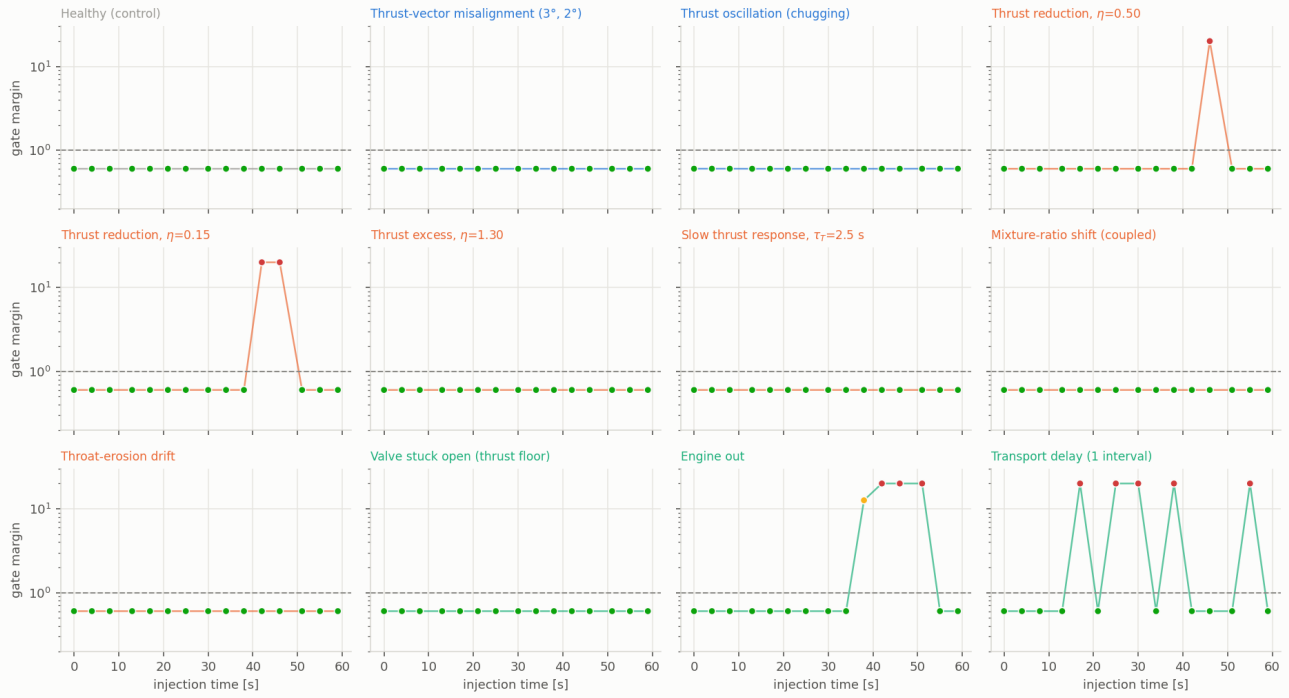


Figure 4: Gate margin against injection time, per fault

4.5 The recoveries themselves

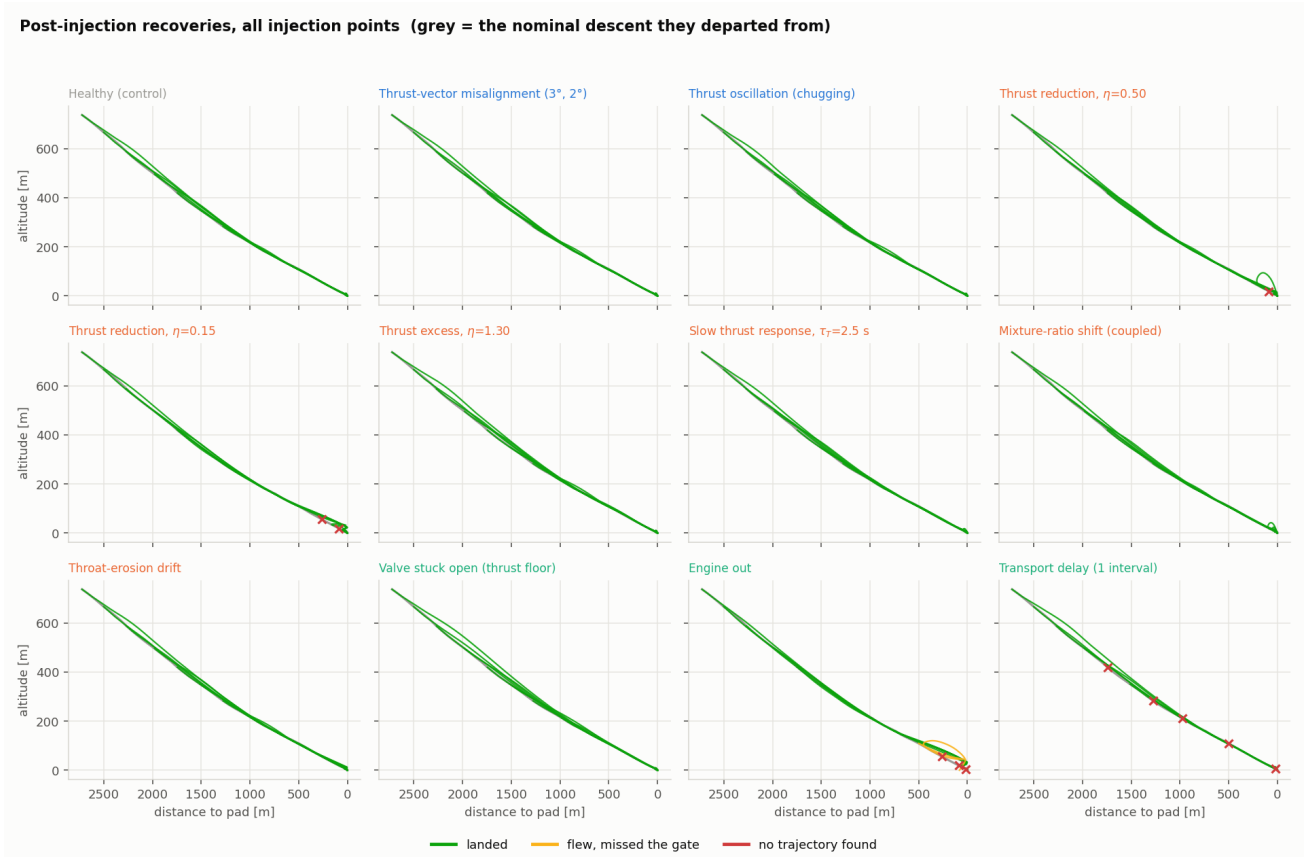


Figure 5: Post-injection recovery trajectories against the nominal

5 What this means

Engine spacing changed the answer, not the fault. At $y_{eng} = 1.50$ m Study G recorded engine-out as unrecoverable from every one of its 50 states. At 0.25 m, inside the gimbal’s roll-trim limit, the same fault on the same vehicle model gives 11/15. Nothing about the failure changed — only whether the surviving engine’s gimbal has the moment arm to trim it. Fault tolerance was decided at the drawing board, not by the controller.

A fault has a survivable window, not a survival probability. The faults that fail do not fail randomly across the descent; they fail *late*. The single number a fault-tolerance budget should carry is the injection point past which recovery stops existing, and that number is a time and an altitude, not a probability.

Altitude is the currency. Every fault that has a boundary has it in the same place — the braking phase, where the vehicle is low, still fast and has committed its remaining altitude to the flare. A fault arriving during the cruise has hundreds of metres to trade for a re-plan; the same fault arriving during the flare has none.

6 Limitations

- **The planner is handed the damaged plant exactly, and instantly.** No detection, isolation, identification or reaction delay. Every number here is an upper bound; Studies D and E measure what the reaction gap costs.
- **Open loop.** Each result is one optimal-control solve, not a closed loop re-solving as it flies. The drift fault in particular is treated more kindly here than a real one would be.

- **One nominal.** All 180 solves depart from a single reference trajectory. A different approach — steeper, faster, offset — would move the boundaries, and this study does not measure how much.
- **The speed schedule is the optimiser’s, not Apollo’s.** The geometry is anchored; the cruise-then-brake profile along it is not how P64 was actually flown.
- **Constant mass,** inherited from the base model, which understates the propellant cost of a partial fault.
- **Local infeasibility is not global.** The NLP is nonconvex; “no trajectory” means IPOPT failed from seven independent seeds (two in the campaign, six in the hardening pass) at up to 1,200 iterations — strong evidence, but not a proof that none exists.
- **15 injection points** resolve the boundary to roughly the spacing between them (4 s); a bisection on onset time would sharpen it, which is what Study D does for a single fault.

7 Reproducing

```
python studies/fault_injection/apollo_nominal.py      # the reference descent
python studies/fault_injection/run_injection_study.py # the campaign
python studies/fault_injection/analyse_injection.py   # figures + headline JSON
python studies/fault_injection/build_report.py        # this document
```